

安装 Docker

安装软件的本质 → apt 仓库信任链 → 境内网络两条链路 → Docker Engine 安装实战

第 2 讲 · 清华 TUNA 系统源 + 阿里云 Docker 源

安装软件本质

核心认知：境内安装要解决两条独立链路

链路 A: apt 下载 .deb 安装包 → download.docker.com 不可达 → 换阿里云 apt 源镜像

链路 B: docker pull 拉取镜像 → docker.io 不可达 → 配 registry-mirrors

📁 链路 A 靠换 **apt 软件源镜像**；链路 B 靠 **registry mirror**——两者互不相干，不能混在一起解决

- 镜像站（阿里云、清华 TUNA、中科大）原样同步官方仓库目录，**GPG 公钥也从镜像域名下载**，信任链不变
- 加速器本质是 daemon 的回退列表：`docker pull` 默认问 `docker.io`，失败后按 `registry-mirrors` 顺序改问镜像站
- ⚠️ 2024 年中起大批公共镜像加速器关停，**镜像清单多已失效**，需配 2~3 个互为备份

安装软件的本质

去掉所有术语，安装一个软件的本质操作只有三件：

1. **复制文件**：可执行文件 → `/usr/bin`，库 → `/usr/lib`，配置 → `/etc`，文档 → `/usr/share/doc`
2. **登记造册**：系统记录装了什么包、什么版本、哪些文件——否则无法升级、卸载、查依赖
3. **运行脚本**：`preinst` / `postinst` / `prerm` / `postrm`——创建用户、注册服务、生成缓存等副作用

所有包管理器，都只是这三件事 的自动化

差别只在「造册格式」与「文件从哪来」。

Ubuntu 装软件的五层全景

层级	工具	管什么	类比
01	dpkg	解包 .deb、复制文件、登记、跑脚本	双手
02	apt	依赖求解、查仓库目录、下载排序	大脑
03	官方仓库 / PPA	软件分发网络（GPG 验签）	批发市场
04	snap / flatpak	依赖自带、沙箱、自动更新	集装箱物流
05	源码编译	自己解决一切	手工作坊

选型原则：优先 apt ，发行版版本太旧或只有 snap 版时上 snap，编译是没办法的办法。

dpkg 与 apt: 双手与大脑

.deb 文件解剖

一个 .deb 本质是 ar 归档，里面两个 tar 包：

```
someapp_1.0_amd64.deb
├── debian-binary          ← 纯文本，只有 "2.0"
├── control.tar.gz         ← 控制信息
│   ├── control           ← 元数据：包名 / 版本 / Depends
│   ├── preinst · postinst ← 安装前后执行的脚本
│   ├── prerm · postrm    ← 卸载前后执行的脚本
│   └── conffiles          ← 用户改过的配置要保留
└── data.tar.gz           ← 真正的文件（tar 内路径 = 安装目标路径）
    ├── usr/bin/someapp
    ├── usr/lib/someapp/...
    └── etc/someapp/config.yaml
```



安装的字面实现：把 data.tar.gz 解压到根目录 / ，文件即各就各位；再执行 postinst。

dpkg 的能力边界

```
sudo dpkg -i someapp.deb      # 装本地 .deb
sudo dpkg -r someapp          # 卸载 (保留配置)
dpkg -l | grep someapp        # 列出已安装
dpkg -L someapp               # 这个包装了哪些文件
dpkg -S /usr/bin/someapp      # 反查文件属于哪个包
```



它不会下载、不会解依赖。依赖缺失就报错退出——1990 年代的 dependency hell 就是这么来的。

apt 的两个大脑

你 → apt (大脑①依赖求解器 + 大脑②仓库客户端) → dpkg (双手)

- 📁 **大脑① 依赖求解器**: 读取 Depends 及版本约束 (`libc6 ≥ 2.34`), 递归求闭包—拓扑排序 + 约束满足
- **大脑② 仓库客户端**: 从 `sources.list` 取"商品目录", `update` 缓存、`install` 提货

推论: 直接 `dpkg -i` 网上下载的 `.deb` = 绕过两个大脑, 容易把系统搞成半依赖状态。装 Docker 走仓库而非裸 `.deb`, 原因在此。

Ubuntu 官方仓库四分区与 PPA

- **main**: 自由软件, 官方维护 (内核、bash、python3)
- **restricted**: 专有驱动, 官方维护 (NVIDIA 驱动)
- **universe**: 自由软件, **社区维护**, 更新可能滞后 (htop、nginx)
- **multiverse**: 版权/法律受限 (某些编解码器)

PPA 的本质和手动加 Docker 仓库**一模一样**: 写一个 `deb ...` 源 + 装一把 GPG 公钥, 只是托管在 Launchpad 上。 `add-apt-repository` 帮你做了这两件事。

⚠ 只加信任作者的 PPA——加 PPA 等于授权对方给你的系统推送任意软件。

生命周期动词表

命令	职责
<code>apt update</code>	刷新所有源的目录缓存， 不装任何东西
<code>apt upgrade</code>	升级，但 不删除任何现有包 （解不开就留旧版）
<code>apt full-upgrade</code>	允许删包解依赖冲突，跨版本升级必用
<code>apt remove</code>	卸程序， 保留 /etc 配置
<code>apt purge</code>	程序 + 配置全删
<code>apt autoremove</code>	清"作为依赖被拉入、现已无人使用"的孤儿包
<code>apt clean</code>	清空 /var/cache/apt/archives/ 的 .deb 缓存
<code>apt policy X</code>	已装版本 vs 仓库可用版本，版本诊断神器

deb vs snap

	.deb (apt)	.snap
依赖	动态链接系统共享库	全部自带，互相隔离
跨发行版	否，与系统版本强绑定	是
沙箱	无	严格，默认访问不了 \$HOME 以外
更新	显式	snapd 自动接管
代价	依赖冲突风险	体积大、库重复、启动略慢

陷阱：Ubuntu 把 firefox 等换成了 snap 版，`apt install firefox` 实际装转接头——用 `which firefox` 和 `snap list` 确认。

apt仓库

体系目录、索引与信任链

仓库物理上就是一个目录树（阿里云 Docker 仓库）

```
https://mirrors.aliyun.com/docker-ce/linux/ubuntu/  
├── dists/                                ← "目录 + 索引"  
│   ├── jammy/  
│   │   ├── InRelease                    ← ★ 全库总目录 + PGP 数字签名  
│   │   └── stable/  
│   │       ├── binary-amd64/  
│   │       │   └── Packages(.gz)        ← ★ 商品清单（纯文本）  
└── pool/                                ← 真正的 .deb 货物  
    └── stable/amd64/*.deb
```



`dists/` 是"目录和索引", `pool/` 是"货物"—两者分离是 Debian 仓库设计的精髓。

InRelease: 总目录 + 签名

-----BEGIN PGP SIGNED MESSAGE-----

Hash: SHA512

Architectures: amd64 arm64 armhf s390x ppc64el ← 有哪些架构可选 (arch= 在选这里的一项)

Components: stable edge test nightly ← 有哪些通道可选 (stable 在选这里的一项)

Suite: jammy ← 服务哪个 Ubuntu 版本 (jammy 填这里)

MD5Sum:

5228944eb5c3cb8d7a321370e4c4e1d6 3080 stable/binary-amd64/Packages

...

-----BEGIN PGP SIGNATURE-----

← Docker 私钥签发, 改一字签名即失效



Packages: 商品清单

每个软件一个条目，没有软件本体，只有"去哪下载、怎么验证"：

```
Package: docker-ce
Version: 28.x.x~ubuntu.22.04~jammy
Depends: containerd.io ( ≥ 1.6.0), ...
Filename: pool/stable/amd64/docker-ce_xxx_amd64.deb    ← apt 安装时拼「仓库URL + Filename」下载
Size: 24300000
SHA256: a7db459b27fa57e73ad6214afc9abeec...          ← 下载后校验完整性
```



配置行 → 目录树映射

```
deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] \  
https://mirrors.aliyun.com/docker-ce/linux/ubuntu \  
jammy stable
```



配置词

落到哪里

deb

声明二进制包仓库（决定找 binary-*/Packages）

URL

所有路径的根

jammy

拼成 dists/jammy/，填错即 404

stable

拼成 dists/jammy/stable/

arch=amd64

只下 binary-amd64/Packages 一份索引

signed-by

验证 dists/jammy/InRelease 签名的公钥

apt update 的四步信任链

1. ① 拼地址下载 `dists/jammy/InRelease`
2. ② 用 `docker.gpg` 公钥验 PGP 签名 — 签名无效立刻中止
3. ③ 按 InRelease 清单下载 `dists/jammy/stable/binary-amd64/Packages.gz` , 哈希校验
4. ④ 缓存到 `/var/lib/apt/lists/` (此后 `install/search` 全查本地缓存)

信任传递链

PGP 签名 — 验签 → InRelease — 哈希 → Packages — 哈希 → 每个 .deb

攻击者黑进镜像站替换 .deb 也没用——没有 Docker 私钥，改不了签名，链条在第一环就断。这就是 signed-by 和装 GPG 公钥的全部意义。

apt install 数据流

```
apt install docker-ce
```

- 查本地缓存 /var/lib/apt/lists/ 的 Packages
- 解析依赖闭包
- 按条目的 Filename 拼下载地址 (仓库URL + Filename)
- 下载 → SHA256 校验 → dpkg 解包安装



先 update 再 install: install 只查本地目录缓存，目录过期就找不到包。

安装Docker命令解析

GPG 公钥管道 + 权限修正

```
curl -fsSL https://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | \  
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg  
  
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```



| 管道：左边命令的标准输出直接喂给右边的标准输入，全程不落中间文件

- `curl -fsSL`： `f` 遇 HTTP 错误即失败（防止把错误页面当密钥）、`s` 静默、`S` 出错时仍要吭声
- `gpg --dearmor`：把带 `BEGIN PGP PUBLIC KEY BLOCK` 外壳的文本公钥转成 apt 用的二进制
- `-o`：输出到文件而非终端；行尾 `\`：命令未完，下一行是延续（纯排版）
- 为什么要 `chmod`：root 创建的文件默认仅所有者可读，apt 验签以普通用户权限读公钥 → 必须所有人可读

仓库配置拼装

```
echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \  
https://mirrors.aliyun.com/docker-ce/linux/ubuntu \  
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```



`$(命令)`：bash 命令替换——先执行括号内命令，把输出填到这里（模板填空）

- `dpkg --print-architecture`：机器自己报 amd64/arm64，不写死
- `. /etc/os-release && echo "$VERSION_CODENAME"`：执行系统元信息文件后取代号变量（jammy），一个模板适配所有 Ubuntu 版本
- `tee`：T 型分叉，把 stdin 一边写文件一边回显；`sudo tee` 才能以 root 写 `/etc` 下的文件（`sudo cmd > file` 里 `sudo` 管不到重定向）
- `> /dev/null`：把 `tee` 的回显丢掉
- 放 `sources.list.d/`：第三方源各占一个“单间”，停用删文件即可

基础工具安装

```
sudo apt install -y ca-certificates curl gnupg
```



-y : 提问自动答 yes (写脚本用; 手动装可去掉)

- ca-certificates : CA 根证书, 没有它 HTTPS 全报不可信
- curl : 命令行下载工具
- gnupg : GPG 工具集

systemctl: 服务的过去与现在

enable --now 的本质：创建符号链接

```
sudo systemctl enable --now docker    # = enable (管以后) + start (管现在)
```



systemd 把系统组织成**依赖图**，节点是 unit（文本配置文件 `/usr/lib/systemd/system/docker.service`）：

```
[Unit]    → 我是谁、依赖谁 (After=network-online.target)
[Service] → 怎么启动我 (ExecStart=/usr/bin/dockerd; Type=notify 启动完主动通知)
[Install] → 我属于哪个目标 (WantedBy=multi-user.target)
```



`enable docker` = 在 `/etc/systemd/system/multi-user.target.wants/` 下创建软链接。只改配置，不启动进程；`disable` 删链接，不影响当前运行中的进程。

systemctl 状态词与 start

- `systemctl is-enabled docker` 的状态词: `enabled` / `disabled` / `static` (无 [Install] 段, 不能 enable) / `masked` (屏蔽, 钩子指向 /dev/null, 连手动 start 都不行)
- **start**: systemd 按 unit 描述 fork 进程执行 ExecStart, 启动完成后标记 active (running)
- 此后进程被 systemd **收养托管**: 崩溃可按 `Restart=` 策略自动拉起

这就是"不要 nohup 手撸后台进程"的原因——手撸的进程没有户口。

配套命令

```
systemctl status docker      # 一眼看全：状态 + PID + 最近日志
systemctl is-active docker   # 只问活没活（脚本判断用）
sudo systemctl reload docker # 重载配置不重启进程（走 ExecReload）
sudo systemctl mask docker   # 屏蔽：谁也启动不了（比 disable 更强）

journalctl -u docker -f      # 服务日志实时滚动（相当于服务的 tail -f）

sudo systemctl daemon-reload # 改了 unit 文件后必须先重扫配置
sudo systemctl restart docker # 再重启才生效—顺序不能反
```



systemd 通过特权 socket 接受管理请求，控制系统服务 = 系统级权限。例外：`systemctl --user` 走用户自己的实例，不需要 sudo（Docker Desktop 用的就是它）。

安装 Docker 完整流程

中国境内

完整流程（上）：步骤 0 ~ 2

```
# 0. 前置
sudo apt update
sudo apt install -y ca-certificates curl gnupg

# 1. GPG 公钥（验签的前提，从国内镜像下载，内容一致）
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# 2. 仓库配置（指向阿里云镜像）
echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
    https://mirrors.aliyun.com/docker-ce/linux/ubuntu \
    $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```



- ① 公钥是验签的前提——必须先从国内镜像下载
- ② 仓库配置指向阿里云镜像，`signed-by` 指向刚才的公钥
- ③ 三个“自动填空”：架构、版本代号、路径
- ④ 全部就绪后，第 3 步才能 `apt update` 找到 `docker-ce`

完整流程（中）：步骤 3 ~ 4

3. 安装（五个包：CLI → 引擎 → 运行时 → 构建 → 编排）

```
sudo apt update
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io \  
    docker-buildx-plugin docker-compose-plugin
```

4. 启动 + 授权

```
sudo systemctl enable --now docker
```

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

📄 不要装 `docker`（无关软件）或 `docker.io`（旧版）

完整流程（下）：步骤 5 ~ 6

```
# 5. 镜像加速器（链路 B）
sudo mkdir -p /etc/docker
sudo tee /etc/docker/daemon.json <<- 'EOF'
{
  "registry-mirrors": [
    "https://docker.m.daocloud.io",
    "https://docker.1ms.run",
    "https://docker.xuanyuan.me"
  ]
}
EOF
sudo systemctl daemon-reload
sudo systemctl restart docker

# 6. 终验（同时验证两条链路）
docker info | grep -A 5 "Registry Mirrors"
docker run hello-world
```

📁 镜像加速器只配 2~3 个互为备份

- 改完 `daemon.json`： `daemon-reload` → `restart`，顺序不能反
- 若有阿里云账号，可在「容器镜像服务 → 镜像加速器」领取专属地址加入 mirrors 列表

实机操作记录（含截图）

实机环境

环境：VMware Workstation 16 Pro（宿主机 Windows）· 虚拟机名 `Ubuntu22.04`

- 4GB 内存 / 2 CPU 内核 / 30GB 磁盘（拆分） / NAT 网络
- ISO: `ubuntu-22.04.5-desktop-amd64.iso`

本机系统源已配清华 TUNA 镜像，Docker 源用阿里云镜像——两条链路分别解决。

8.1 · VMware Workstation 安装 Ubuntu 22.04

以下 16 步为 VMware 图形界面操作，每步对应一张实机截图。



步骤 1 · 打开新建虚拟机向导

主页点击「创建新的虚拟机」，选择**典型（推荐）**配置。



步骤 2 · 安装来源：稍后安装操作系统

先建空硬盘再挂 ISO，可控性更强。



步骤 3 · 选择客户机操作系统

Linux → Ubuntu 64 位。

WORKSTATION 16 PRO™



创建新的虚拟机



打开虚拟机



连接远程服务器

新建虚拟机向导

命名虚拟机

您希望该虚拟机使用什么名称？

虚拟机名称(V):

Ubuntu22.04

位置(L):

D:\Documents\Virtual Machines\Ubuntu22.04

浏览(B)...

在“编辑”>“首选项”中可更改默认位置。

< 上一步(B)

下一步(N) >

取消

步骤 4 · 命名虚拟机

名称 Ubuntu22.04，位置 D:\Documents\Virtual Machines\Ubuntu22.04。



连接远程服务器

×

磁盘大小为多少?

磁盘大小为多少?

最大磁盘大小 (GB)(S):

针对 Ubuntu 64 位的建议大小: 20 GB

- 将虚拟磁盘拆分成多个文件(M)

拆分磁盘后，可以更轻松地在计算机之间移动虚拟机，但可能会降低大容量磁盘的性能。

帮助

 $\leq 1 - \frac{1}{2^k} (B)$

下一步(N) >

取消

30 GB，选择将虚拟磁盘拆分成多个文件（便于移动，代价是大容量下性能略降）。



步骤 6 · 确认配置

汇总核对——Ubuntu 64 位、30GB 拆分磁盘、4GB 内存、NAT 网络、2 个 CPU 内核，点击「完成」。

Ubuntu22.04

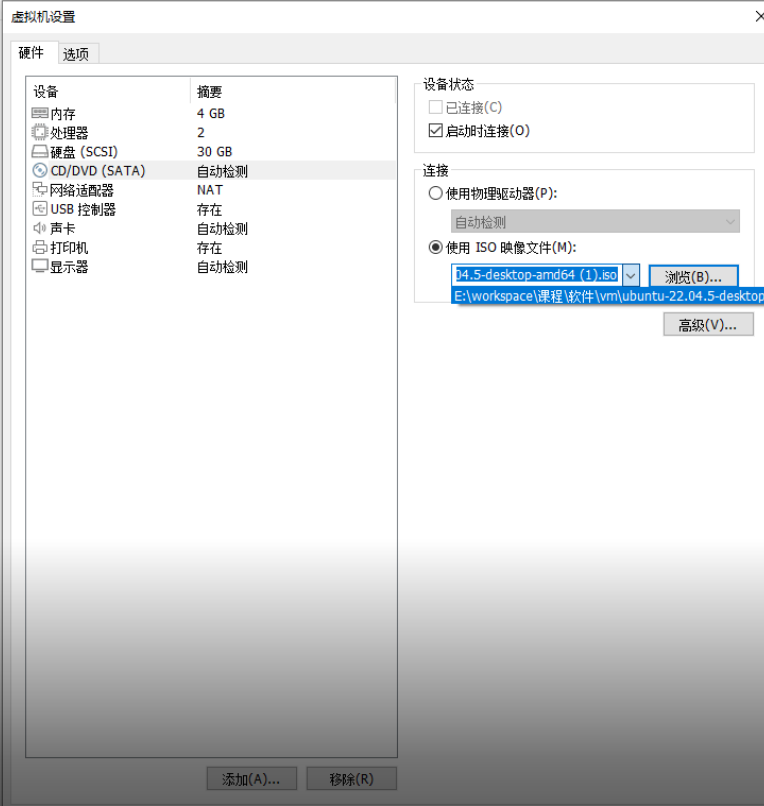
[开启此虚拟机](#)[编辑虚拟机设置](#)

设备

内存	4 GB
处理器	2
硬盘 (SCSI)	30 GB
CD/DVD (SATA)	自动检测
网络适配器	NAT
USB 控制器	存在
声卡	自动检测
打印机	存在
显示器	自动检测

描述

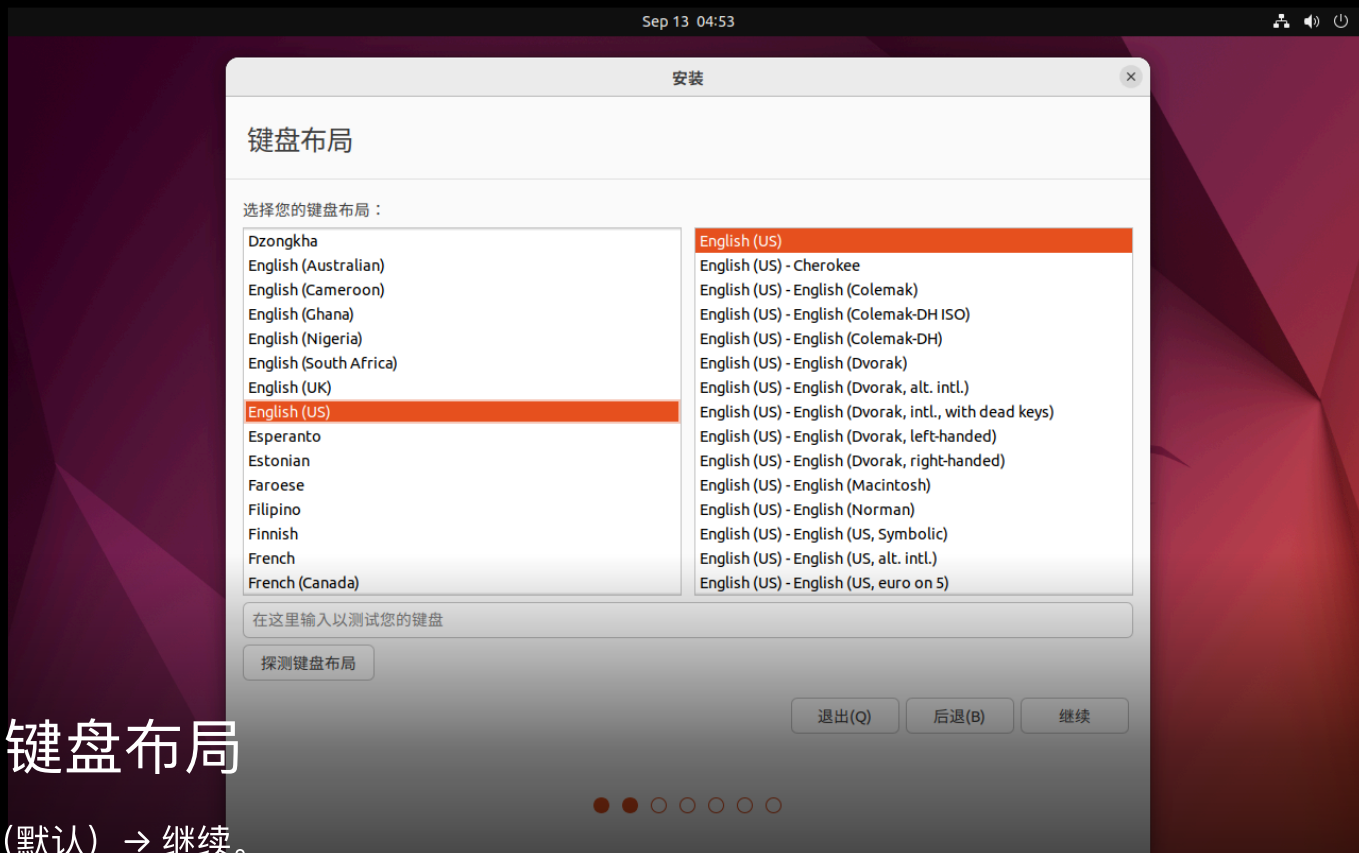
在此处键入对该虚拟机的描述。



步骤 7 · 挂载 ISO

虚拟机设置 → CD/DVD (SATA) → 使用 ISO 映像文件，选中 E:\ubuntu-22.04.5-desktop-amd64.iso，勾选「启动时连接」。

状态: 已关机
配置文件: D:\Documents\Virtual Machines\Ubuntu22.04\Ubuntu22.04.vmx
硬件兼容性: Workstation 16.x 虚拟机
主 IP 地址: 网络信息不可用



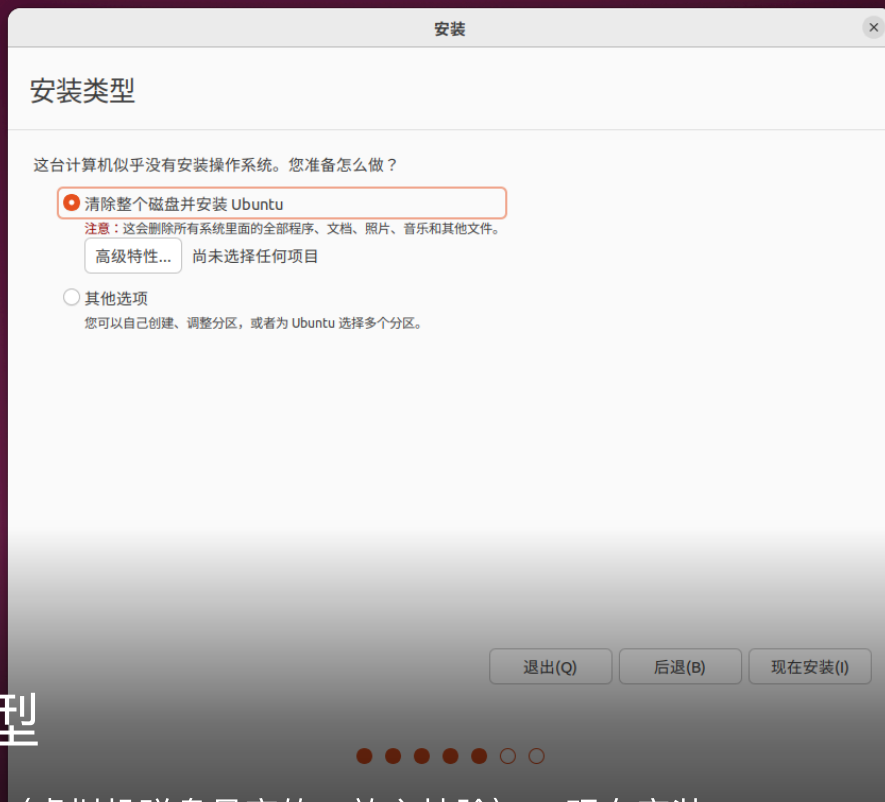
步骤 8 · 键盘布局

English (US) (默认) → 继续。



步骤 9 · 更新和其他软件

选择**最小安装**（网络浏览器和基本工具）——Docker 学习环境够用，体积更小；勾选「安装 Ubuntu 时下载更新」，不勾第三方软件。



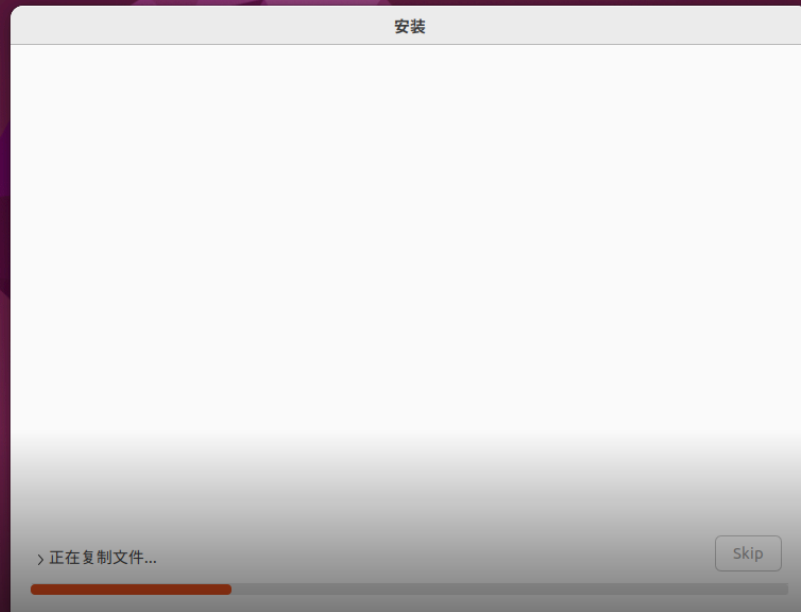
步骤 10 · 安装类型

清除整个磁盘并安装 Ubuntu（虚拟机磁盘是空的，放心抹除）→ 现在安装。



步骤 11 · 时区

Shanghai (UTC+8)。



步骤 12 · 复制文件

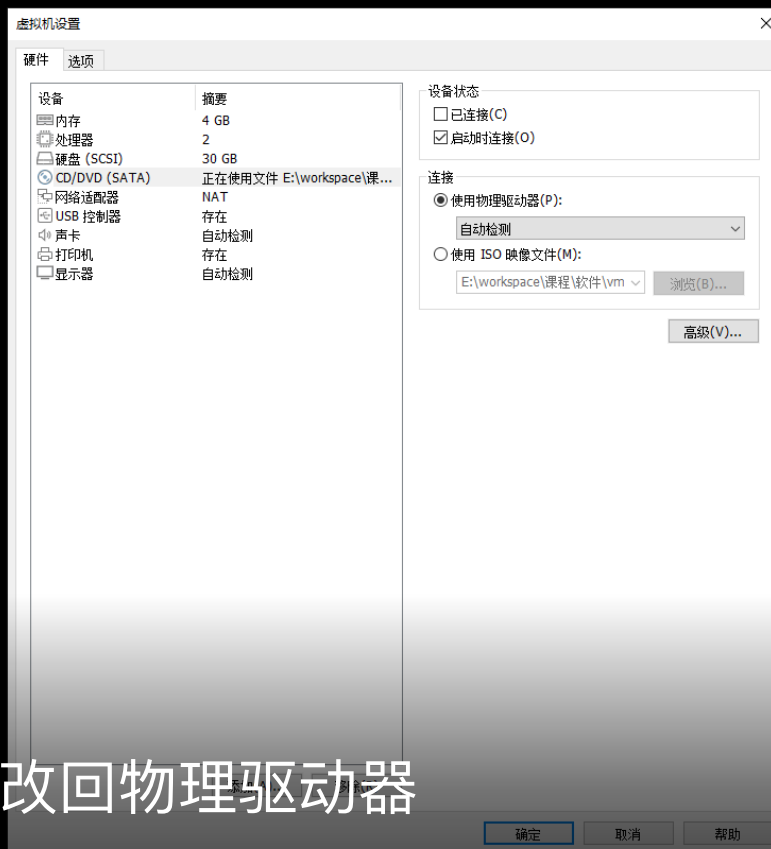
等待安装程序复制文件（约数十分钟）。



步骤 13 · 安装完成——关键一步

点击「现在重启」。

⚠️ **关键：** 重启前把 CD/DVD 从 ISO 改回物理驱动器（或断开连接），否则虚拟机会再次从 ISO 引导、循环进入安装界面。



步骤 14 · CD/DVD 改回物理驱动器

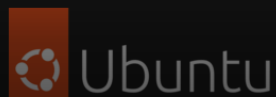
重启前必须改回，否则虚拟机会循环进入安装界面。

9月13日 13:12

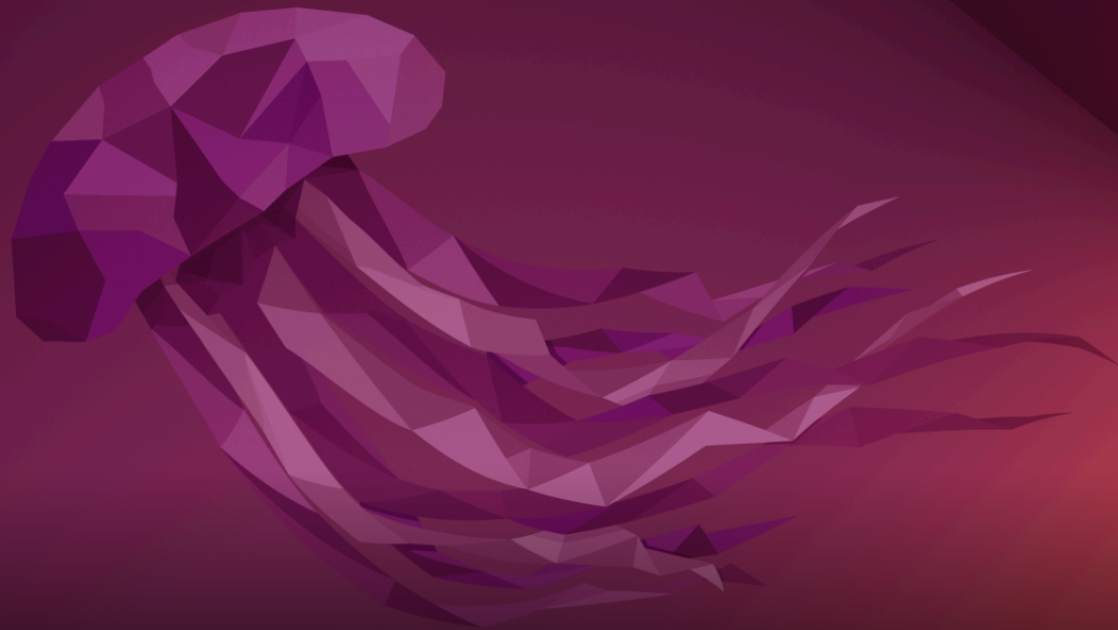


未列出?

步骤 15 · 从硬盘启动并登录



进入登录界面，选择用户 **xujc** 输入密码。



步骤 16 · 进入桌面

Ubuntu 22.04 桌面（Jellyfish 壁纸）。基础系统安装完成。



主目录

单击虚拟屏幕
以发送按键

VMware Tools 具有很多功能，能改善鼠标移动性、视频和性能。请登录客户机操作系统，单击“安装 Tools”。

安装 Tools

以后提醒我

不要提醒

入定向到该虚拟机。请在虚拟机内部单击或按 Ctrl+G。



8.2 · Ubuntu 终端安装 Docker

本机系统源已配清华 TUNA 镜像，Docker 源用阿里云镜像（两条链路分别解决）。

```
xujc@xujc-virtual-machine:~$ sudo apt update
```

```
[sudo] xujc 的密码:
```

```
命中:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
```

```
命中:2 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy InRelease
```

```
命中:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates InRelease
```

```
命中:4 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-backports InRelease
```

```
正在读取软件包列表... 完成
```

```
正在分析软件包的依赖关系树... 完成
```

```
正在读取状态信息... 完成
```

```
有 426 个软件包可以升级。请执行 'apt list --upgradable' 来查看它们。
```

```
xujc@xujc-virtual-machine:~$
```

步骤 1 · 刷新索引

sudo apt update, 命中清华源四个组件, 提示 426 个包可升级。

正在读取状态信息... 完成

有 426 个软件包可以升级。请执行 'apt list --upgradable' 来查看它们。

```
xujc@xujc-virtual-machine:~$ sudo apt install -y ca-certificates curl gnupg
```

正在读取软件包列表... 完成

正在分析软件包的依赖关系树... 完成

正在读取状态信息... 完成

将会同时安装下列软件:

```
dirmngr gnupg-l10n gnupg-utils gpg gpg-agent gpg-wks-client gpg-wks-server gpgconf gpgsm gpgv libcurl4
```

建议安装:

```
tor parcimonie xloadimage scd daemon
```

下列【新】软件包将被安装:

```
curl
```

下列软件包将被升级:

```
ca-certificates dirmngr gnupg gnupg-l10n gnupg-utils gpg gpg-agent gpg-wks-client gpg-wks-server gpgconf gpgsm gpgv libcurl4
```

升级了 13 个软件包, 新安装了 1 个软件包, 要卸载 0 个软件包, 有 413 个软件包未被升级。

需要下载 486 kB/2,874 kB 的归档。

解压缩后会消耗 412 kB 的额外空间。

获取:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main amd64 libcurl4 amd64 7.81.0-1ubuntu1.27 [292 kB]

获取:2 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main amd64 curl amd64 7.81.0-1ubuntu1.27 [194 kB]

已下载 486 kB, 耗时 1秒 (449 kB/s)

正在预设软件包 ...

(正在读取数据库 ... 系统当前共安装有 181686 个文件和目录。)

准备解压 .../00-gpg-wks-client_2.2.27-3ubuntu2.5_amd64.deb ...

正在解压 gpg-wks-client (2.2.27-3ubuntu2.5) 并覆盖 (2.2.27-3ubuntu2.1) ...

准备解压 .../01-dirmngr_2.2.27-3ubuntu2.5_amd64.deb ...

正在解压 dirmngr (2.2.27-3ubuntu2.5) 并覆盖 (2.2.27-3ubuntu2.1) ...

准备解压 .../02-gpg-wks-server_2.2.27-3ubuntu2.5_amd64.deb ...

正在解压 gpg-wks-server (2.2.27-3ubuntu2.5) 并覆盖 (2.2.27-3ubuntu2.1) ...

准备解压 .../03-gnupg-utils_2.2.27-3ubuntu2.5_amd64.deb ...

正在解压 gnupg-utils (2.2.27-3ubuntu2.5) 并覆盖 (2.2.27-3ubuntu2.1) ...

准备解压 .../04-gpg-agent_2.2.27-3ubuntu2.5_amd64.deb ...

正在解压 gpg-agent (2.2.27-3ubuntu2.5) 并覆盖 (2.2.27-3ubuntu2.1) ...

准备解压 .../05-gpg_2.2.27-3ubuntu2.5_amd64.deb ...

步骤2: 装基础工具

sudo apt install -y cacertificates curl gnupg, apt 自动解依赖 (升级 13 个包、新装 curl)。

```
0 added, 0 removed; done.
```

```
Running hooks in /etc/ca-certificates/update.d...
```

```
done.
```

```
xujc@xujc-virtual-machine:~$ ls /etc/apt/keyrings/
```

```
xujc@xujc-virtual-machine:~$ ls -al /etc/apt/keyrings/
```

```
总计 8
```

```
drwxr-xr-x 2 root root 4096 4月 8 2022 .
```

```
drwxr-xr-x 8 root root 4096 9月 13 13:03 ..
```

```
xujc@xujc-virtual-machine:~$ ls -al /etc/apt/
```

```
总计 44
```

```
drwxr-xr-x 8 root root 4096 9月 13 13:03 .
```

```
drwxr-xr-x 129 root root 12288 9月 13 14:47 ..
```

```
drwxr-xr-x 2 root root 4096 9月 13 13:03 apt.conf.d
```

```
drwxr-xr-x 2 root root 4096 4月 8 2022 auth.conf.d
```

```
drwxr-xr-x 2 root root 4096 4月 8 2022 keyrings
```

```
drwxr-xr-x 2 root root 4096 9月 11 2024 preferences.d
```

```
-rw-rw-r-- 1 root root 2822 9月 13 13:03 sources.list
```

```
drwxr-xr-x 2 root root 4096 4月 8 2022 sources.list.d
```

```
drwxr-xr-x 2 root root 4096 9月 11 2024 trusted.gpg.d
```

```
xujc@xujc-virtual-machine:~$ sudo install -m 0755 -d /etc/apt/keyrings
```

```
xujc@xujc-virtual-machine:~$ ls -al /etc/apt/
```

```
总计 44
```

```
drwxr-xr-x 8 root root 4096 9月 13 13:03 .
```

```
drwxr-xr-x 129 root root 12288 9月 13 14:47 ..
```

```
drwxr-xr-x 2 root root 4096 9月 13 13:03 apt.conf.d
```

```
drwxr-xr-x 2 root root 4096 4月 8 2022 auth.conf.d
```

```
drwxr-xr-x 2 root root 4096 4月 8 2022 keyrings
```

```
drwxr-xr-x 2 root root 4096 9月 11 2024 preferences.d
```

```
-rw-rw-r-- 1 root root 2822 9月 13 13:03 sources.list
```

```
drwxr-xr-x 2 root root 4096 4月 8 2022 sources.list.d
```

```
drwxr-xr-x 2 root root 4096 9月 11 2024 trusted.gpg.d
```

```
xujc@xujc-virtual-machine:~$
```

步骤3 创建密钥目录

`sudo install -m 0755 -d /etc/apt/keyrings; ls -al /etc/apt/` 观察目录结构 (keyrings、sources.list.d、trusted.gpg.d 并列)。

```
xujc@xujc-virtual-machine:~$ curl -fsSL https://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | \
> sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
xujc@xujc-virtual-machine:~$ ls /etc/apt/keyrings/
docker.gpg
xujc@xujc-virtual-machine:~$ ls -l /etc/apt/keyrings/
总计 4
-rw-r--r-- 1 root root 2760  9月 13 14:51 docker.gpg
xujc@xujc-virtual-machine:~$ sudo chmod a+r /etc/apt/keyrings/docker.gpg
xujc@xujc-virtual-machine:~$ ls -l /etc/apt/keyrings/
总计 4
-rw-r--r-- 1 root root 2760  9月 13 14:51 docker.gpg
xujc@xujc-virtual-machine:~$
```

步骤 4 · 下载 GPG 公钥

`curl -fsSL .../gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg`，随后 `chmod a+r`；`ls -l` 确认权限为 `-rw-r--r--`。

```
xujc@xujc-virtual-machine:~$ dpkg --print-architecture
```

```
amd64
```

```
xujc@xujc-virtual-machine:~$ cat /etc/os-release
```

```
PRETTY_NAME="Ubuntu 22.04.5 LTS"
```

```
NAME="Ubuntu"
```

```
VERSION_ID="22.04"
```

```
VERSION="22.04.5 LTS (Jammy Jellyfish)"
```

```
VERSION_CODENAME=jammy
```

```
ID=ubuntu
```

```
ID_LIKE=debian
```

```
HOME_URL="https://www.ubuntu.com/"
```

```
SUPPORT_URL="https://help.ubuntu.com/"
```

```
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
```

```
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
```

```
UBUNTU_CODENAME=jammy
```

```
xujc@xujc-virtual-machine:~$ ^[[200~echo \
```

```
> "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
```

```
> https://mirrors.aliyun.com/docker-ce/linux/ubuntu \
```

```
> ${. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
```

```
> sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
echo: 未找到命令
```

```
xujc@xujc-virtual-machine:~$ cat /etc/apt/sources.list.d/docker.list
```

```
deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
```

```
https://mirrors.aliyun.com/docker-ce/linux/ubuntu \
```

```
${. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
xujc@xujc-virtual-machine:~$ cat /etc/apt/sources.list.d/docker.list
```

```
deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://mirrors.aliyun.com/docker-ce/linux/ubuntu \
```

```
$(. /etc/os-release && echo "$VERSION_CODENAME") stable
```

步骤 5: 写仓库配置 (真实踩坑)

先 `dpkg --print-architecture (amd64)`、`cat /etc/os-release (jammy)` 确认两个自动填空项; 写 `docker.list` 并 `cat` 验证内容正确。

⚠ 本步真实踩坑: 首次执行时命令首混入了终端控制字符, 报 `echo: 未找到命令` ——重新干净输入后成功。

```
xujc@xujc-virtual-machine:~$ sudo apt update # 刷新目录缓存, 让 apt 认识 Docker 仓库
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io \
```

```
docker-buildx-plugin docker-compose-plugin
```

```
获取:1 https://mirrors.aliyun.com/docker-ce/linux/ubuntu jammy InRelease [48.5 kB]
```

```
获取:2 https://mirrors.aliyun.com/docker-ce/linux/ubuntu jammy/stable amd64 Packages [87.0 kB]
```

```
命中:3 http://security.ubuntu.com/ubuntu jammy-security InRelease
```

```
命中:4 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy InRelease
```

```
命中:5 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates InRelease
```

```
命中:6 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-backports InRelease
```

```
已下载 135 kB, 耗时 1秒 (95.9 kB/s)
```

```
正在读取软件包列表... 完成
```

```
正在分析软件包的依赖关系树... 完成
```

```
正在读取状态信息... 完成
```

```
有 413 个软件包可以升级。请执行 'apt list --upgradable' 来查看它们。
```

```
正在读取软件包列表... 完成
```

```
正在分析软件包的依赖关系树... 完成
```

```
正在读取状态信息... 完成
```

```
将会同时安装下列软件:
```

```
docker-ce-rootless-extras git git-man liberror-perl pigz
```

```
建议安装:
```

```
cgroupfs-mount | cgroup-lite docker-model-plugin git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn
```

```
下列【新】软件包将被安装:
```

```
containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin git git-man liberror-perl pigz
```

```
升级了 0 个软件包, 新安装了 10 个软件包, 要卸载 0 个软件包, 有 413 个软件包未被升级。
```

```
需要 17.6 MB 的存储空间。
```

```
解压后还会消耗 403 MB 的额外空间。
```

```
获取:1 https://mirrors.aliyun.com/docker-ce/linux/ubuntu jammy/stable amd64 containerd.io amd64 2.3.5-1-ubuntu.22.04~jammy [22.4 MB]
```

```
获取:2 http://mirrors.tuna.tsinghua.edu.cn/docker-ce/linux/ubuntu jammy/stable amd64 pigz amd64 2.16-1 [63.6 kB]
```

```
获取:3 http://mirrors.tuna.tsinghua.edu.cn/docker-ce/linux/ubuntu jammy/stable amd64 liberror-perl all 0.1.0-2 [20.5 kB]
```

```
获取:4 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main amd64 git amd64 1:2.34.1-1ubuntu1.17 [954 kB]
```

```
获取:5 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main amd64 git amd64 1:2.34.1-1ubuntu1.17 [3,174 kB]
```

```
获取:6 https://mirrors.aliyun.com/docker-ce/linux/ubuntu jammy/stable amd64 docker-ce-cli amd64 5:29.8.0-1-ubuntu.22.04~jammy [17.6 MB]
```

```
获取:7 https://mirrors.aliyun.com/docker-ce/linux/ubuntu jammy/stable amd64 docker-ce amd64 5:29.8.0-1-ubuntu.22.04~jammy [24.3 MB]
```

步骤6: 刷新索引 + 安装五个包

sudo apt update 成功从阿里云拉取 jammy InRelease 和 stable amd64 Packages; apt install 五个包, 混合从阿里云 Docker 源与清华 Ubuntu 源下载全部依赖。

```
xujc@xujc-virtual-machine:~$ sudo systemctl enable --now docker
```

```
Synchronizing state of docker.service with SysV service script with /lib/systemd/systemd-sysv-install.
```

```
Executing: /lib/systemd/systemd-sysv-install enable docker
```

```
xujc@xujc-virtual-machine:~$
```

步骤 7 · 启动服务

`sudo systemctl enable --now docker`，钩子挂上且进程拉起。

```
xujc@xujc-virtual-machine:~$ docker ps
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
xujc@xujc-virtual-machine:~$ sudo usermod -aG docker $USER
xujc@xujc-virtual-machine:~$ newgrp docker
xujc@xujc-virtual-machine:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

```
xujc@xujc-virtual-machine:~$
```

步骤 8 · 用户授权

首次 `docker ps` 报 `permission denied` (socket 权限); `usermod -aG docker $USER + newgrp docker` 后正常输出表头。

```
xujc@xujc-virtual-machine:~$ docker run hello-world
```

```
Unable to find image 'hello-world:latest' locally
```

```
docker: Error response from daemon: failed to resolve reference "docker.io/library/hello-world:latest": failed to do request: Head "https://registry-1.docker.io/v2/library/hello-world/manifests/latest": dial tcp 210.56.51.193:443: connect: connection refused
```

```
Run 'docker run --help' for more information
```

```
xujc@xujc-virtual-machine:~$
```

步骤 9 · 链路 B 卡点

docker run hello-world 报 connection refused —daemon 直连 registry-1.docker.io 被拒。

```
docker: Error response from daemon: failed to resolve reference "docker.io/library/hello-world:latest": failed to do request: Head "https://registry-1.docker.io/v2/library/hello-world/manifests/latest":
dial tcp 210.56.51.193:443: connect: connection refused
```

Run 'docker run --help' for more information

```
xujc@xujc-virtual-machine:~$ sudo mkdir -p /etc/docker
```

```
xujc@xujc-virtual-machine:~$ sudo mkdir -p /etc/docker
```

```
sudo tee /etc/docker/daemon.json <<-'EOF'
```

```
{
  "registry-mirrors": [
    "https://docker.m.daocloud.io",
    "https://docker.1ms.run",
    "https://docker.xuanyuan.me"
  ]
}
```

EOF

```
sudo systemctl daemon-reload
```

```
sudo systemctl restart docker
```

```
{
  "registry-mirrors": [
    "https://docker.m.daocloud.io",
    "https://docker.1ms.run",
    "https://docker.xuanyuan.me"
  ]
}
```

```
xujc@xujc-virtual-machine:~$ docker info
```

```
Registry Mirrors:
```

```
https://docker.m.daocloud.io/
```

```
https://docker.1ms.run/
```

```
https://docker.xuanyuan.me/
```

```
Live Restore Enabled: false
```

```
Firewall Backend: iptables
```

```
xujc@xujc-virtual-machine:~$
```

步骤 10 配镜像加速器

写 daemon.json (daocloud / 1ms.run / xuanyuan 三个互为备份), daemon-reload + restart docker, docker info 确认三个镜像源已生效。

xujc@xujc-virtual-machine: ~

```
Live Restore Enabled: false
Firewall Backend: iptables
xujc@xujc-virtual-machine:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
d5e71e642bf5: Download complete
4f55086f7dd0: Pull complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest
```

Hello from Docker!

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share this message on your favorite social media platform with the Docker ID:

<http://www.docker.com/>

For more examples and ideas, visit:

<http://id.docker.com/>

```
xujc@xujc-virtual-machine:~$
```

步骤 1.1 · 终验成功

docker run hello-world 拉取成功，输出 Hello from Docker!——两条链路全部打通。

附录 A · 排错速查表 (1/2)

链路 A 与 apt 相关

排错速查表（1/2）：链路 A 与 apt 相关

现象	定位	解法
<code>apt update</code> 卡在 docker 源	链路 A：源地址不通	检查 <code>docker.list</code> 域名、网络
GPG error / NO_PUBKEY	公钥没装或不可读	重跑公钥安装两步，确认 <code>chmod a+r</code>
404 Not Found (jammy InRelease)	仓库代号填错	确认 <code>VERSION_CODENAME</code> 与系统匹配
Unable to locate package <code>docker-ce</code>	没跑 update 或源没生效	安装前先 <code>sudo apt update</code>
<code>echo</code> ：未找到命令	复制命令带入 <code>\$</code> 或控制 字符	检查命令首字符； <code>cat docker.list</code> 验证

附录 A · 排错速查表 (2/2)

链路 B 与运行相关

排错速查表 (2/2): 链路 B 与运行相关

现象	定位	解法
<code>docker run</code> 拉镜像超时 / refused	链路 B: Docker Hub 不可达	配置 <code>daemon.json</code> 的 <code>registry-mirrors</code>
<code>permission denied</code> (docker.sock)	用户组未生效	<code>newgrp docker</code> 或注销重新登录
拉 <code>gcr.io</code> / <code>ghcr.io</code> 失败	加速器只代理 Docker Hub	前缀法: <code>docker pull docker.m.daocloud.io/gcr.io/xxx</code> 再 <code>docker tag</code> 改回
重启后又进安装界面	ISO 仍挂在光驱	CD/DVD 改回物理驱动器或断开
改 <code>unit</code> 文件 / <code>daemon.json</code> 后不生效	配置没重扫	<code>daemon-reload</code> 再 <code>restart</code> , 顺序不能反

附录 B · 全流程命令速览

全流程命令速览

完整命令见第 7 章「中国境内安装 Docker 完整流程」，可直接整段复制执行。

- 两条独立链路：**apt 源** 解决下载 .deb，**registry-mirrors** 解决拉镜像
- 信任链：**PGP 签名** → **InRelease** → **Packages** → **每个 .deb**，链条第一环最硬
- 安装本质：**复制文件 + 登记造册 + 运行脚本**
- 三句口令：`apt update` → `apt install docker-ce ...` → `systemctl enable --now docker`

小结

- 从第一性原理理解 apt：仓库 = 目录 + 索引 + 签名，安装 = 查单 → 下载 → 校验 → 解包
- 境内两条链路分开解决：清华 TUNA 系统源 + 阿里云 Docker 源 + 镜像加速器
- 完整跑通 VMware + Ubuntu 22.04 + Docker Engine（11 张实机截图全程验证）
- 排错速查表在手，`daemon-reload` 再 `restart`，顺序不能反

Q&A